



សាកលវិទ្យាល័យ អាស៊ី អឺរ៉ុប
មហាវិទ្យាល័យវិទ្យាសាស្ត្រ និងបច្ចេកវិទ្យា

មុខវិជ្ជា ៖ Advance C++ Programming
សាស្ត្រាចារ្យ ៖ ស៊ុន ម៉ាឡែន
លេខទូរស័ព្ទ ០១៦៥៨៥៧៨០

<https://elearning.aeu.cloud>



មាតិកា

1. Inheritance

1.1. ស្វែងយល់អំពី Inheritance

1.2. គុណសម្បត្តិ និង គុណវិបត្តិ Inheritance

1.3. ឧទាហរណ៍ និង Coding



១.១.ស្វែងយល់អំពី Inheritance

- **Inheritance** គឺនៅក្នុងភាសា C++ (Object-Oriented Programming) Inheritance ជា ការស្នងមរតក។

➤ Inheritance ត្រូវបានបែងចែកជា ៥ ប្រភេទសំខាន់ៗ៖

- 1) **Single Inheritance** គឺ Class កូន (Derived Class) មួយ ស្នងមរតកផ្ទាល់ពី Class ឪពុក (Base Class) តែមួយគត់។ **ឧទាហរណ៍៖** Parent -> Child
- 2) **Multilevel Inheritance Class** គឺ មួយស្នងមរតកពី Class មួយទៀត ដែល Class នោះក៏ជាកូនរបស់ Class មួយផ្សេងទៀតដែរ (ដូចជា ជីតា, ឪពុក, កូន)។ **ឧទាហរណ៍៖** Grandparent -> Parent -> Child
- 3) **Multiple Inheritance Class** គឺ កូនមួយ ស្នងមរតកពី Class ឪពុកម្តាយច្រើនក្នុងពេលតែមួយ (C++ គាំទ្រលក្ខណៈនេះ ប៉ុន្តែភាសាខ្លះដូចជា Java ឬ C# មិនគាំទ្រផ្ទាល់ទេព្រោះវាអាចបង្កបញ្ហា Diamond Problem)។ **ឧទាហរណ៍៖** Father + Mother -> Child

១.១.ស្វែងយល់អំពី Inheritance

- 4) Hierarchical Inheritance គឺ Class ឌីពុកតែមួយ មាន Class កូនច្រើនមកស្ដង់មរតកយកទៅប្រើប្រាស់ រៀងៗខ្លួន។ **ឧទាហរណ៍៖** Animal -> Dog, Cat, Cow
- 5) Hybrid Inheritance គឺការស្ដង់មរតកចម្រុះ ឬ រួមបញ្ចូលគ្នារវាង Inheritance លើសពី ២ ប្រភេទឡើង ទៅ (**ឧទាហរណ៍៖** ការបូកបញ្ចូលគ្នារវាង Hierarchical និង Multiple)។
- (Hierarchical): Class Animal គឺជា Base Class ធំ ដែលត្រូវបានស្ដង់មរតកដោយ Class Human និង Class Dog (Animal តែមួយ បែកមែកធាងជាពីរកូន)។
 - (Multiple): Class Child ស្ដង់មរតកពីប្រភពពីរក្នុងពេលតែមួយ គឺពី Class Human (ដែលជាបំណាមជីវជា Animal) និង Class Father។

១.១. ស្វែងយល់អំពី Inheritance Syntax

```
class BaseClass {};
```

Single Inheritance

```
class DerivedClass : access_specifier BaseClass {};
```

```
class ClassA { ... };
```

Multilevel Inheritance Class

```
class ClassB : public ClassA { ... };
```

```
class ClassC : public ClassB { ... };
```

```
class Base1 { ... };
```

Multiple Inheritance Class

```
class Base2 { ... };
```

```
class Derived : public Base1, public Base2 { ... };
```

```
class Parent { ... };
```

Hierarchical Inheritance

```
class Child1 : public Parent { ... };
```

```
class Child2 : public Parent { ... };
```

```
class A { ... };
```

Hybrid Inheritance

```
class B : public A { ... };
```

```
class C : public A { ... };
```

```
class D : public B, public C { ... };
```

១.២.គុណសម្បត្តិ និង គុណវិបត្តិ Inheritance(ត)

គុណសម្បត្តិ Inheritance	គុណវិបត្តិ Inheritance
Reusability (ការយកកូដមកប្រើឡើងវិញ): អនុញ្ញាតឱ្យ Class កូដយកទិន្នន័យ ឬមុខងារពី Class ដ៏ពុកមកប្រើ ដោយមិនបាច់សរសេរកូដដដែលៗឡើងវិញ។	Tight Coupling (ការភ្ជាប់គ្នាស្អិតពាក់): Class ដ៏ពុក និង Class កូដមានទំនាក់ទំនងគ្នាខ្លាំង។ ប្រសិនបើមានការកែប្រែ ឬលុបកូដនៅក្នុង Class ដ៏ពុក វាអាចធ្វើឱ្យកូដនៅក្នុង Class កូដរំខាន ឬ error ទាំងអស់។
Easy Maintenance (ងាយស្រួលថែទាំកូដ): បើចង់កែប្រែ ឬបន្ថែមមុខងារទូទៅ គឺកែតែនៅលើ Base Class (Class ដ៏ពុក) តែមួយកន្លែង វានឹងរាលដាលទៅ Class កូដទាំងអស់ដោយស្វ័យប្រវត្ត។	Decreased Execution Speed (ល្បឿនដំណើរការថយចុះ): ការហៅប្រើប្រាស់មុខងារឆ្លងកាត់ Class ច្រើនគង់ (ជាពិសេសក្នុង Multilevel Inheritance) អាចធ្វើឱ្យកម្មវិធីស៊ីពេលវេលា (Execution time) ជាងធម្មតាបន្តិច។
Extensibility (លទ្ធភាពពង្រីកកូដ): ងាយស្រួលក្នុងការបង្កើតមុខងារថ្មីៗបន្ថែមនៅលើ Class កូដ ដោយគ្រាន់តែស្នងមរតកពី Class ចាស់ រួចសរសេរហួសពីហ្នឹង (Overriding/Extending)។	Increased Complexity (ភាពស្មុគស្មាញនៃកូដ): បើប្រើប្រាស់វាជ្រុលហួសហេតុ (ដូចជាបង្កើតកូដចៅច្រើនជាន់ពេក ឬប្រើ Multiple/Hybrid) វានឹងធ្វើឱ្យកូដពិបាកអាន ពិបាកយល់ និងពិបាករកកំហុស (Debugging)។

គុណសម្បត្តិ Inheritance	គុណវិបត្តិ Inheritance
Polymorphism Support (គាំទ្រពហុភាព): វាជាគ្រឹះដ៏សំខាន់ក្នុងការបង្កើត Method Overriding ដែលអនុញ្ញាតឱ្យប្រព័ន្ធគ្រប់គ្រង Object ផ្សេងៗគ្នា តាមរយៈ Interface ឬ Base Class តែមួយ។	Memory Wastage (ការខ្ចះខ្ចាយមេម៉ូរី): នៅពេលបង្កើត Object របស់ Class កូន វានឹងទាញយក ទិន្នន័យទាំងអស់ពី Class ឪពុកមកជាមួយ (ទោះបីជា មុខងារខ្លះ Class កូនមិនត្រូវការប្រើក៏ដោយ)។
Reusability (ការយកកូដមកប្រើឡើងវិញ): អនុញ្ញាតឱ្យ Class កូនយកទិន្នន័យ ឬមុខងារពី Class ឪពុកមកប្រើ ដោយមិនបាច់សរសេរកូដដដែលៗឡើង វិញ។	Tight Coupling (ការភ្ជាប់គ្នាស្អិតពាក់): Class ឪពុក និង Class កូនមានទំនាក់ទំនងគ្នាខ្លាំង។ ប្រសិនបើមានការកែប្រែ ឬលុបកូដនៅក្នុង Class ឪពុក វាអាចធ្វើឱ្យកូដនៅក្នុង Class កូនរំខាន ឬ error ទាំងអស់។
Easy Maintenance (ងាយស្រួលថែទាំកូដ): បើចង់កែប្រែ ឬបន្ថែមមុខងារទូទៅ គឺកែតែនៅលើ Base Class (Class ឪពុក) តែមួយកន្លែង វានឹងរាលដាល ទៅ Class កូនៗទាំងអស់ដោយស្វ័យប្រវត្ត។	Decreased Execution Speed (ល្បឿនដំណើរការយឺត): ការហៅប្រើប្រាស់មុខងារឆ្លងកាត់ Class ច្រើនតង់ (ជា ពិសេសក្នុង Multilevel Inheritance) អាចធ្វើឱ្យកម្មវិធីស៊ី ពេលវេលា (Execution time) ជាងធម្មតាបន្តិច។

១.៣. ឧទាហរណ៍ និង Coding

➤ Project Example: Example គេប្រើប្រាស់ Inheritance មានដូចជា ៖

- Banking System => Example : Account
- Game Development => Example : Character, Entity
- E-Commerce / ERP System => Example : Person, User
- Graphics / UI Frameworks => Example : Shape

Ex1 : Single Inheritance (ការសងមរតកទោល)

```
1  #include <iostream>
2  using namespace std;
3
4  class Animal {
5  public:
6      void eat() { cout << "Animal is eating..." << endl; }
7  };
8
9  class Dog : public Animal {
10 public:
11     void bark() { cout << "Dog is barking..." << endl; }
12 };
13
14 int main() {
15     Dog myDog;
16     myDog.eat();
17     myDog.bark();
18     return 0;
19 }
```

```
Animal is eating
Dog is barking..
```

Ex2 : Multilevel Inheritance (ការសង្ខេបមរតកច្រើនជាន់)

```
#include <iostream>
using namespace std;

class Automobile {
public:
    void start() { cout << "Engine started." << endl; }
};

class Car : public Automobile {
public:
    void drive() { cout << "Car is moving." << endl; }
};

class SportsCar : public Car {
public:
    void turbo() { cout << "Turbo mode activated!" << endl; }
};

int main() {
    SportsCar myCar;
    myCar.start();
    myCar.drive();
    myCar.turbo();
    return 0;
}
```

Ex3 : Multiple Inheritance (ការស្នងមរតកពីប្រភពច្រើន)

```
1  #include <iostream>
2  using namespace std;
3
4  class Camera {
5  public:
6      void takePhoto() { cout << "Photo taken." << endl; }
7  };
8
9  class Phone {
10 public:
11     void makeCall() { cout << "Calling..." << endl; }
12 };
13
14 class SmartPhone : public Camera, public Phone {
15 public:
16     void browseInternet() { cout << "Browsing web..." << endl; }
17 };
18
19 int main() {
20     SmartPhone myPhone;
21     myPhone.makeCall();
22     myPhone.takePhoto();
23     myPhone.browseInternet();
24     return 0;
25 }
```

Calling...

Photo taken.

Browsing web...

Ex4 : Hierarchical Inheritance (ការស្នងមរតកបែប ហ៊ានុក្រម)

```

1  #include <iostream>
2  using namespace std;
3
4  class Shape {
5  public:
6      void draw() { cout << "Drawing a shape." << endl; }
7  };
8
9
10 class Rectangle : public Shape {
11 public:
12     void areaRect() { cout << "Calculating Rectangle Area." << endl; }
13 };
14
15 class Circle : public Shape {
16 public:
17     void areaCircle() { cout << "Calculating Circle Area." << endl; }
18 };
19
20 int main() {
21     Rectangle rect;
22     Circle circ;
23
24     rect.draw();
25     circ.draw();
26     return 0;
27 }

```

Drawing a shape.
Drawing a shape.

Ex5 : Hybrid Inheritance (ការសង្ខេបមរតកចម្រុះ)

```

1  #include <iostream>
2  using namespace std;
3
4  // Base Class ទី១
5  class Animal {
6  public:
7      void eat() {
8          cout << "Eating food..." << endl;
9      }
10 };
11
12 // Hierarchical Inheritance
13 class Human : public Animal {
14 public:
15     void speak() {
16         cout << "Speaking a language..." << endl;
17     }
18 };
19
20 // Base Class ទី២
21 class Father {
22 public:
23     void advice() {
24         cout << "Giving advice and ..." << endl;
25     }
26 };
27
28 // Multiple Inheritance: Child ស្ថាបនាមរតកពីទាំង Human និង Father
29 //Hybrid Inheritance
30 class Child : public Human, public Father {
31 public:
32     void play() {
33         cout << "Child is playing..." << endl;
34     }
35 };
36
37 int main() {
38     Child obj;
39
40     cout << "---- Child Object Accessing Capabilities ----" << endl;
41     obj.play();
42     obj.advice();
43     obj.speak();
44     obj.eat();
45
46     return 0;
47 }

```

```

---- Child Object Accessing Capabilities ----
Child is playing...
Giving advice and ...
Speaking a language...
Eating food...

```



សាកលវិទ្យាល័យ អាស៊ី អឺរ៉ុប

ASIA EURO UNIVERSITY

សិក្សាស្រាវជ្រាវ សង្គមសេចក្តីសុខ

អរគុណ

សម្រាប់ការយកចិត្តទុកដាក់

<https://elearning.aeu.cloud>